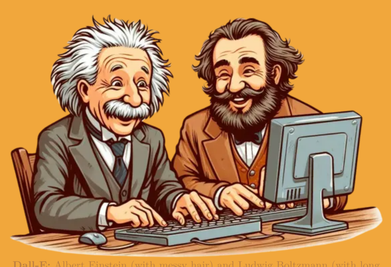


Dall-E: Albert Einstein (with messy hair) and Ludwig Boltzmann (with long beard) happily solving equations together on a computer. Cartoon. Old style.

SymBoltz.jl: symbolic-numeric, approximation-free and differentiable Boltzmann code

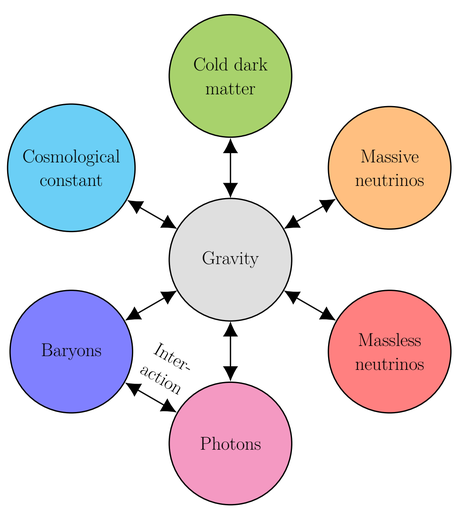
Herman Sletmoen – University of Oslo

New features

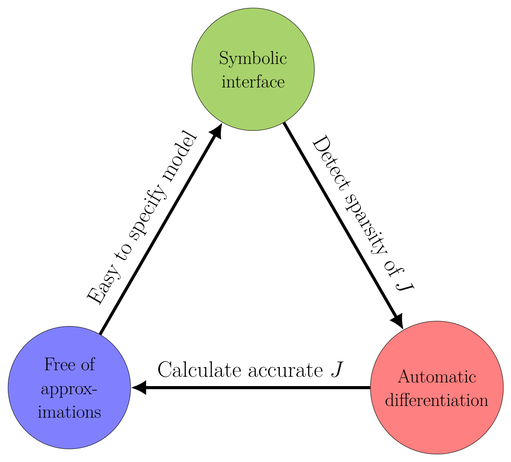
- **Symbolic-numeric architecture:** Represents equations symbolically, and generates fast numerical code from them. Combines the powers of symbolic equation modeling and efficient numerical computing into one integrated framework for easier cosmological modeling.
- **Modular physical structure:** Structured around physical components (e.g. gravity and particle species). Easily replace one component with another without modifying the entire model. Build a library of small self-contained physical components, and join any of them into full cosmological models. Accomodates extended models in a way that scales in model space. Computational stages (background, perturbations, …) should be detected automatically from interdependencies between the equations.
- **Approximation-free:** Integrates the full (stiff) Einstein-Boltzmann equations at all times with a wide selection of modern (implicit) solvers. No tight coupling, ultra-relativistic, radiation streaming or other approximations. Makes code simpler and extended models easier to implement.
- **Differentiable:** Compute $\partial(\text{output})/\partial(\text{input})$ for any input and output (e.g. $\partial P(k)/\partial\Omega_{m0}$) with automatic differentiation (exact numerical differentiation of arbitrarily complicated functions). Important for accurate Jacobians for implicit ODE solvers, and for gradient-based numerical methods, like efficient many-parameter inference methods (e.g. HMC/NUTS), machine learning and optimization problems.
- **Convenient:** Eliminates chores one must do when extending traditional codes. Focus on *modeling*.

Architecture

SymBoltz.jl relies on ModelingToolkit.jl for symbolic modeling and conversion to efficient numerical code, and on DifferentialEquations.jl for ODE integration and access to stiff solvers. We want to take advantage of and contribute to this growing symbolic-numeric modeling library for the benefit of all sciences.



Structure: For optimal modularity, SymBoltz is structured according to the *physical* layout of cosmological models. Components can be added, replaced or modified independently. The best *computational* structure (e.g. background → thermodynamics → perturbations) is found internally.



Synergy of features: For example, automatic differentiation helps to calculate the ODE Jacobian J robustly, which is needed by implicit solvers to integrate ODEs without approximations, which makes it easier to write down models in a simple symbolic form, which can be used to detect the sparsity pattern of J, which in turn increases the efficiency of automatic differentiation, …. This makes a stronger self-reinforcing design.

Goals

- **Create an integrated symbolic-numeric Einstein-Boltzmann modeling *environment*** (not "just" a solver). Grow into a full linear + non-linear framework including N-body simulations?
- **Make it easier for modelers to build modified models** by avoiding complicating approximations and using mixed symbolic-numeric computing to automate chores and provide helpful utilities. The goal is to **let the modeler simply write down *one single set* of Einstein-Boltzmann equations for their cosmological model**. SymBoltz should *automatically* detect the structure in the equations, split it up into stages (background → thermodynamics → perturbations → …), spline functions between each stage, generate suitable initial conditions, change to a suitable independent variable, generate code and solve the model, compute and plot derived quantities, and so on.
- **Design a structure that is modular in terms of *physical* components** and scales better in model space. Rethink the traditional Boltzmann solver layout to avoid duplicating large monolithic codes for every combination of models/extensions, which often grow into mutually incompatible forks.
- **Enable powerful gradient-based optimization/MCMC methods** (e.g. HMC/NUTS) by being fast and differentiable. Important for many-parameter inference from next-generation surveys!

Implementation status (complete, in progress and planned)

- Baryons + photons (RECFAST recombo.)
- Cold dark matter
- Massless and massive neutrinos
- Cosmological constant
- Quintessence
- w_0w_a dark energy
- General Relativity
- Brans-Dicke gravity
- Curved geometry
- Symbolic interface
- Approximation-free (no TCA, RSA, UFA, …)
- Forward-mode automatic differentiation
- Matter power spectrum
- CMB power spectrum (TT, TE, EE)
- Automatic stage separation (backgr. → pert.)
- Automatic splining of ODE unknowns
- Automatic recomputation of ODE observeds
- Automatic interpolation in (τ, k) of all variables
- Automatic change of variables ($\frac{\mathrm{d}}{\mathrm{d}\tau} \to \frac{\mathrm{d}}{\mathrm{d}\ln a}$)
- Automatic series expansion of initial conditions
- Automatic parameter assignment hooks
- Shooting method for boundary conditions
- Plot recipes (for convenient inspection)
- Performance (isolate symbolics from numerics)
- Extensive documentation pages
- Continuous integration and testing
- Comparison with CLASS for ΛCDM
- Non-linear: HALOFIT/HMCODE
- Non-linear: generate N-body problems
- Reverse-mode automatic differentiation
- Automatic unit conversion utilities
- Symbolic tensor manipulation
- GPU parallellization
- Better representation of interactions
- Publication paper
- **Ideas/contributions are very welcome!**

Try it! Feedback?

Easy installation and extensive documentation:

github.com/hersle/SymBoltz.jl

SymBoltz.jl is in development. Feedback is appreciated!
Interested? Questions? Suggestions? Contributions?
Star it! 🌟 Open an issue or pull request! 👷

Code demonstration (as of latest version)

Install and launch `julia`. Load some packages and build a standard ΛCDM model:

```
using SymBoltz, Unitful, UnitfulAstro, ModelingToolkit, ForwardDiff, Plots
M = ΛCDM()
```

This is a purely *symbolic* model representation. It can be interactively inspected and modified. Every physical component lives in a 1-letter subcomponent: the metric g, gravity G, photons γ, massless neutrinos ν, massive neutrinos h, cold dark matter c, baryons b and the cosmological constant Λ. There is no separation between the background and perturbations here. For example, we can view the equations in the gravity component:

```
equations(M.G)
```

$$\frac{\mathrm{d}a\left(t\right)}{\mathrm{d}t} = \sqrt{\frac{8}{3}\left(a\left(t\right)\right)^4\rho\left(t\right)\pi} \tag{1}$$

$$\Delta\left(t\right) = \frac{\mathrm{d}}{\mathrm{d}t}\frac{\mathrm{d}a\left(t\right)}{\mathrm{d}t} + \frac{-\left(\frac{\mathrm{d}a(t)}{\mathrm{d}t}\right)^2\left(1+\frac{-3P(t)}{\rho(t)}\right)}{2a\left(t\right)} \tag{2}$$

$$\frac{\mathrm{d}\Phi\left(t\right)}{\mathrm{d}t}\epsilon = \left(\frac{-k^2\Phi\left(t\right)}{3\mathscr{E}\left(t\right)} + \frac{-\frac{4}{3}\left(a\left(t\right)\right)^2\delta\rho\left(t\right)\pi}{\mathscr{E}\left(t\right)} - \mathscr{E}\left(t\right)\Psi\left(t\right)\right)\epsilon \tag{3}$$

$$k^2\left(-\Psi\left(t\right)+\Phi\left(t\right)\right)\epsilon = 12\left(a\left(t\right)\right)^2\Pi\left(t\right)\pi\epsilon \tag{4}$$

Next, we can map parameters to some values and compile a *numerical* problem from the symbolic system. This is where "the magic happens": the symbolic system is checked for consistency, broken down into background and perturbation stages, splines are set up to pass the solution of one stage to the next, and so on.

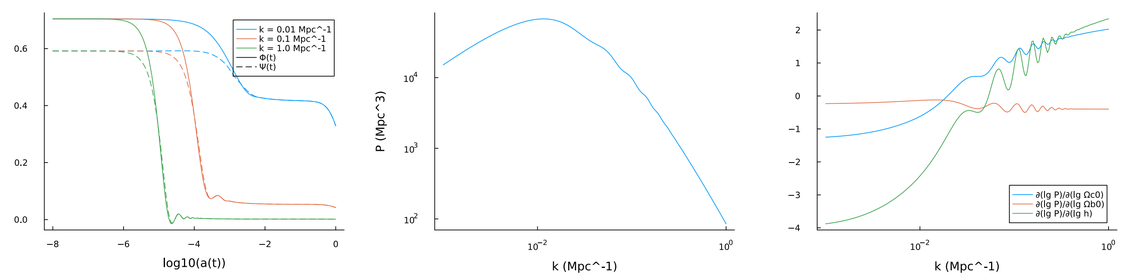
```
params = Dict(
    M.γ.T₀ => 2.7,  M.ν.Neff => 3.0,  M.g.h      => 0.7,
    M.c.Ω₀ => 0.27, M.b.Ω₀   => 0.05, M.b.rec.Yp => 0.25,
    M.I.As => 2e-9, M.I.ns   => 0.95, M.h.m      => 0.06u"eV/c^2/kg"+0
)
prob = CosmologyProblem(M, params) # symbolic-to-numeric compilation
```

Now we can solve the problem. The resulting solution object can be indexed with any symbolic expression to get its numerical value (it is automatically computed from the ODE state vector). We can also calculate derived quantities, like the matter power spectrum. Gradients can be obtained with automatic differentiation.

```
ks = [1e-2, 1e-1, 1e0] / u"Mpc" # wavenumbers
sol = solve(prob, ks) # solution object
p1 = plot(sol, ks, log10(M.g.a), [M.g.Φ, M.g.Ψ]) # use included plot recipe

ks = 10 .^ range(-3, 0, length=200) / u"Mpc" # wavenumbers for matter power spectrum
function Pk(θ) # define P(k; θ) as a function of some independent parameters θ
    newprob = remake(prob, Dict(M.c.Ω₀ => θ[1], M.b.Ω₀ => θ[2], M.g.h => θ[3]))
    sol = solve(newprob, ks; ptopts = (reltol = 1e-3,))
    return spectrum_matter(sol, ks)
end
θ = [params[M.c.Ω₀], params[M.b.Ω₀], params[M.g.h]]
Ps = Pk(θ)
dlgPs = ForwardDiff.jacobian(lgθ -> log10.(Pk(10 .^ lgθ) / u"Mpc^3"), log10.(θ))
p2 = plot(ks, Ps; xlabel="k", ylabel="P", xaxis=:log, yaxis=:log, label=nothing)
p3 = plot(ks, dlgPs; xlabel="k", xaxis=:log, label="∂(lg P)/∂(lg ".*["Ωc0)" "Ωb0)" "h)"])

p = plot(p1, p2, p3; size=(1600,400), layout=(1,3), margin=(8,:mm), bg=false, grid=false)
```



Finally, let us implement a new species for w_0w_a dark energy (as if it is not already available in the code).

```
using SymBoltz: t, k, ϵ, D, O # use variables, derivative and perturbation operator
g = M.g # get metric of original model
pars = @parameters w₀ wₐ cₛ² Ω₀ # create parameters
vars = @variables ρ(t) P(t) w(t) δ(t) θ(t) σ(t) # create variables
eqs = [ # specify equations
    O(1)(w ~ w₀ + wₐ * (1 - g.a)) # equation of state
    O(1)(ρ ~ 3/(8*Num(π))*Ω₀ * g.a^(-3*(1+w₀+wₐ))) # energy density
    O(1)(P ~ w * ρ) # pressure
    O(ϵ)(D(δ) ~ -(1+w)*(θ-3*g.Φ) - 3*g.ℰ*(cₛ²-w)*δ) # overdensity
    O(ϵ)(D(θ) ~ -g.ℰ*(1-3*w)*θ - D(w)/(1+w)*θ + k^2*(cₛ²/(1+w)*δ + g.Ψ)) # momentum
    O(ϵ)(σ ~ 0) # shear stress
]
initialization_eqs = [ # initial conditions
    O(ϵ)(δ ~ -3/2 * (1+w) * g.Ψ)
    O(ϵ)(θ ~ 1/2 * (k^2/g.ℰ) * g.Ψ)
]
description = "w₀wₐ (CPL) dynamical dark energy"
@named X = ODESystem(eqs, t, vars, pars; initialization_eqs, description)

M = ΛCDM(Λ = X; name = :w₀wₐCDM) # recreate ΛCDM model, but replace CC by w0wa
params = merge(params, Dict(M.X.w₀ => -0.9, M.X.wₐ => 0.2, M.X.cₛ² => 1.0))
prob = CosmologyProblem(M, params) # create new problem
sol = solve(prob, ks) # solve new problem; then continue as before ...
```

We only had to write down the equations; not modify any internal code. Everything (e.g. background and perturbation quantities) is defined in the same place to encourage modularity. Similar implementations in other codes can require many more changes and in many different places, effectively forking and fragmenting the code. Proceed as before to inspect and solve the new model. See the documentation for more examples!